

ՀԱՅԱՍՏԱՆԻ ՀԱՆՐԱՊԵՏՈՒԹՅԱՆ ԿՐԹՈՒԹՅԱՆ ԵՎ ԳԻՏՈՒԹՅԱՆ ՆԱԽԱՐԱՐՈՒԹՅՈՒՆ
ՀԱՅԱՍՏԱՆԻ ԱԶԳԱՅԻՆ ՊՈԼՅՏԵԽՆԻԿԱԿԱՆ ՀԱՄԱԼՅԱՐԱՆԻ ԳՅՈՒՄՐՈՒ ՄԱՍՆԱՃՅՈՒՂ

Լաբորատոր աշխատանք

Առարկա՝ Օբյեկտ կողմնորոշված ծրագրավորում

Խումբ՝ ԳԻԿ 519Բ

Կուրս՝ 2-րդ

Ուսանող՝ Խաչատրյան Անի

Դասախոս՝ Սոնյա Սահակյան

Գյումրի – 2026

Լաբորատոր աշխատանք 3

Կառուցվածքային տիպեր C++ լեզվում

Առաջադրանքները նախատեսված են կառուցվածքների, կառուցվածքների զանգվածների, ցուցիչների և ֆունկցիաներին փոխանցման եղանակների ամրապնդման համար: Յուրաքանչյուր լուծում պետք է բաժանել տրամաբանական ֆունկցիաների և պահպանել տվյալների ստուգման պահանջները:

Ընդհանուր պահանջներ

1. Յուրաքանչյուր խնդրում սահմանել առանձին struct տիպ և օգտագործել դաշտերի իմաստալից անգլերեն անուններ:
2. Մուտքագրումը, հաշվարկը, որոնումը կամ փոփոխումը կազմակերպել առնվազն երկու առանձին ֆունկցիայով:
3. Միայն ընթերցող ֆունկցիաներում կիրառել const հղում կամ const զանգված: Փոփոխող ֆունկցիաներում կիրառել հղում կամ ցուցիչ:
4. Ստուգել քանակների, գների, գնահատականների և այլ թվային տվյալների թույլատրելի սահմանները:
5. Դասակարգելիս փոխանակել ամբողջ կառուցվածքային տարրը:
6. Արտածումը ձևավորել ընթեմնելի վերնագրերով: Կողը լրացնել կարծ մեկնաբանություններով:

Ջանձնման ստուգաթերթ

- Ծրագիրը կոմպիլացվում և աշխատում է առանց սխալների:
- Բոլոր մուտքային սահմանները ստուգված են:
- Ֆունկցիաների պարամետրերը ընտրված են ըստ նշանակության:
- Դինամիկ հիշողությունը, եթե կիրառվել է, ազատված է:
- Օրինակային մուտքն ու ելքը կցված են աշխատանքին:

Ավտոմեքենաների ընտրություն

Car կառուցվածքում պահել մակնիշ, տարեթիվ, գին, վաճառ: Մուտքագրել 9 մեքենա: Ֆունկցիային փոխանցել զանգվածը և գնային սահմանը, ապա արտածել համապատասխան մեքենաները՝ վաճառի աճման կարգով: Գտնել ամենանոր մեքենան սահմանված բյուջեի ներսում:

Պարտադիր կիրառել: կառուցվածքների զանգված, առնվազն երկու ֆունկցիա, որոնում կամ դասակարգում:

Արդյունք: Ծրագիրը պետք է արտածի մշակված գրառումները և խնդրում պահանջված ամփոփ արդյունքը:

```
#include <iostream>
#include <string>
#include <vector>
#include <limits>
#include <algorithm>
using namespace std;

// 1. Car կառուցվածքի սահմանում՝ իմաստալից անգլերեն անուններով
struct Car {
    string brand;
    int year;
    double price;
    double mileage;
};

// Ֆունկցիա՝ մեկ մեքենայի տվյալները ստուգումով մուտքագրելու
// համար (փոխտխող ֆունկցիա՝ հղումով)
void inputCar(Car& car, int index) {
    cout << "\n--- Մուտքագրեք մեքենա #" << index + 1 << " ---\n";

    cout << "Մակնիշ: ";
    cin >> car.brand;

    // Արտադրման տարեթվի ստուգում
    do {
        cout << "Տարեթիվ (1900-2026): ";
        cin >> car.year;
    } while (car.year < 1900 || car.year > 2026);

    // Գնի ստուգում (պետք է լինի ոչ բացասական)
    do {
        cout << "Գին ($): ";
        cin >> car.price;
    } while (car.price < 0);

    // Վազքի ստուգում (պետք է լինի ոչ բացասական)
    do {
```

```

        cout << "Վազք (կմ): ";
        cin >> car.mileage;
    } while (car.mileage < 0);
}

```

// 2. Մեքենաների զանգվածի մուտքագրում

```

void inputAllCars(Car cars[], int size) {
    for (int i = 0; i < size; ++i) {
        inputCar(cars[i], i);
    }
}

```

// Ֆունկցիա՝ մեկ մեքենայի տվյալները տպելու համար (միայն ընթերցող՝ const հղումով)

```

void printCar(const Car& car) {
    cout << "Մակնիշ: " << car.brand
        << " | Տարեթիվ: " << car.year
        << " | Գին: $" << car.price
        << " | Վազք: " << car.mileage << " կմ/ժ";
}

```

// 3. Ֆունկցիա՝ զանգվածը գտելու, վազքով տեսակավորելու և ամենանորը գտնելու համար

// Օգտագործում է const հղում զանգվածի համար

```

void processAndDisplayCars(const Car cars[], int size, double maxPrice) {
    vector<Car> filteredCars;

```

// Չտու՛մ ենք մեքենաները՝ ըստ բյուջեի

```

    for (int i = 0; i < size; ++i) {
        if (cars[i].price <= maxPrice) {
            filteredCars.push_back(cars[i]);
        }
    }
}

```

```

    if (filteredCars.empty()) {
        cout << "\n[!] Սահմանված բյուջեին ($" << maxPrice << ")
համապատասխանող մեքենա չի գտնվել:\n";
        return;
    }
}

```

// Դասակարգում (տեսակավորում) ըստ վազքի աճման կարգի՝
փոխանակելով ամբողջ struct-ը

```
for (size_t i = 0; i < filteredCars.size() - 1; ++i) {  
    for (size_t j = 0; j < filteredCars.size() - i - 1; ++j) {  
        if (filteredCars[j].mileage > filteredCars[j + 1].mileage) {  
            // Ամբողջ կառուցվածքի փոխանակում  
            Car temp = filteredCars[j];  
            filteredCars[j] = filteredCars[j + 1];  
            filteredCars[j + 1] = temp;  
        }  
    }  
}
```

// Արտածում ենք բյուջեի մեջ գտնվող մեքենաները՝ վազքի աճման
կարգով

```
cout <<  
"n=====n";  
cout << " ԲՅՈՒՋԵԻՆ ($" << maxPrice << ") ՀԱՄԱՊԱՏԱՍԽԱՆՈՂ  
ՄԵՔԵՆԱՆԵՐ (ՎԱՉՔԻ ԱՃՄԱՆ ԿԱՐԳՈՎ)n";  
cout <<  
"=====n";  
for (const auto& car : filteredCars) {  
    printCar(car);  
}
```

// Որոնում ենք ամենանոր մեքենան (ամենամեծ տարեթվով) բյուջեի
ներսում

```
const Car* newestCar = &filteredCars[0];  
for (size_t i = 1; i < filteredCars.size(); ++i) {  
    if (filteredCars[i].year > newestCar->year) {  
        newestCar = &filteredCars[i];  
    }  
}
```

// Ամփոփ արդյունքի արտածում

```
cout << "n-----n";  
cout << "  ԱՄԵՆԱՆՈՐ ՄԵՔԵՆԱՆ ՍԱՀՄԱՆՎԱԾ ԲՅՈՒՋԵԻ ՆԵՐՍՈՒՄՈՐn";  
cout << "-----n";  
printCar(*newestCar);  
cout << "-----n";
```

```
}
```

```
int main() {  
    const int CAR_COUNT = 9;  
    Car cars[CAR_COUNT]; // Կառուցվածքների զանգված (9 մեքենա)  
  
    cout << "=== Մեքենաների տվյալների մուտքագրում(" << CAR_COUNT  
<< " ՄԵՔԵՆԱ) ===\n";  
    inputAllCars(cars, CAR_COUNT);  
  
    // Գնային սահմանի (բյուջեի) մուտքագրում  
    double maxBudget;  
    do {  
        cout << "\nՄուտքագրեք առավելագույն բյուջեն ($): ";  
        cin >> maxBudget;  
    } while (maxBudget < 0);  
  
    // Մշակում և արտածում  
    processAndDisplayCars(cars, CAR_COUNT, maxBudget);  
  
    return 0;  
}
```